Credit Risk Modeling

Minh Duc Nguyen

2025-04-09

```r
library(data.table)
library(dplyr)
library(tidyr)
library(ggplot2)
library(scorecard)
library(caret)
library(pROC)
library(rpart)
library(rpart.plot)
library(xgboost)
library(ranger)
library(SHAPforxgboost)
library(pdp)

DATA_DIR    <- "E:/NEU SLIDE/DATA SCIENCE"
INPUT_FILE  <- file.path(DATA_DIR, "credit_cleaned.csv")
LGD_ASSUMED <- 0.45
SAMPLE_SIZE <- 200000
SEED        <- 123
```

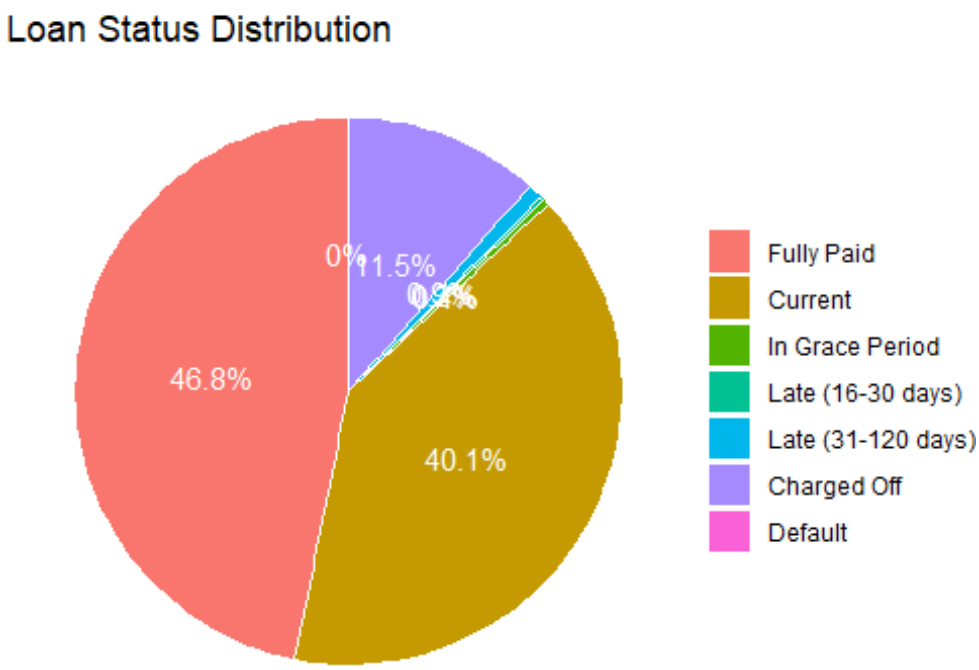
1. Data Loading & Sampling

```r
set.seed(SEED)
credit <- fread(INPUT_FILE)
cat("Bad rate (original):", round(mean(credit$loan_status %in%
c(3,4,5,8,9))*100,2), "%\n")

## Bad rate (original): 13.04 %

credit <- credit[, .SD[sample(.N, min(round(SAMPLE_SIZE * .N / nrow(credit)),
.N))],
                 by = loan_status]

cat("Rows after sampling:", nrow(credit), "\n")

## Rows after sampling: 200000

cat("Bad rate after sampling:", round(mean(credit$loan_status %in%
c(3,4,5,8,9))*100,2), "%\n")

## Bad rate after sampling: 13.04 %

status_labels <- c("Fully Paid","Current","In Grace Period",
                   "Late (16-30 days)","Late (31-120 days)","Charged
Off","Default")
```

```
invisible(credit[, loan_status := factor(loan_status,
levels=c(1,2,3,4,5,8,9),
                                          labels=status_labels)])
invisible(credit[, loan_status_bin := factor(
  fifelse(loan_status %in% c("Fully Paid","Current"), "Good","Bad"),
  levels = c("Good","Bad")
)])
```

2. Loan Status Distribution

```
loan_dist <- credit[, .N, by=loan_status][, pct := N/sum(N)*100]

ggplot(loan_dist, aes(x="", y=pct, fill=loan_status)) +
  geom_col(width=1, color="white") +
  coord_polar(theta="y") +
  geom_text(aes(label=paste0(round(pct,1),"%")),
            position=position_stack(vjust=0.5), color="white", size=4) +
  labs(title="Loan Status Distribution") +
  theme_void() + theme(legend.title=element_blank())
```



3. Feature Preparation

Split X and Y into two separate datasets:

- **data_raw**: original values -> Decision Tree, XGBoost, RF, Scorecard
- **data_scaled**: standardized -> Logistic Regression only

```
# Variables only available AFTER loan default -> data leakage
# These cannot be known at the time of loan origination
leakage_vars <- c(
  # Post-default payment & recovery variables
  "recoveries", "collection_recovery_fee",
  "last_pymnt_amnt", "last_pymnt_d", "last_pymnt_d_since_issue",
  "out_prncp", "out_prncp_inv",
  "total_pymnt", "total_pymnt_inv",
  "total_rec_prncp", "total_rec_int", "total_rec_late_fee",
  # Post-default bureau pulls
  "last_credit_pull_d", "last_credit_pull_d_since_issue",
  "last_fico_range_high", "last_fico_range_low",
  # Hardship & restructuring flags (set only after borrower is in distress)
  "hardship_flag", "has_hardship", "ever_hardship",
  # Payment plan (set only after delinquency)
  "pymnt_plan", "has_payment_plan",
  # Debt settlement (set only after default)
  "debt_settlement_flag"
)

drop_cols <- intersect(
  c("loan_status", "loan_status_bin", "policy_code",
"home_ownership_tempMissing",
    leakage_vars),
  names(credit)
)

removed_leakage <- intersect(leakage_vars, names(credit))
if (length(removed_leakage) > 0) {
  cat("Removing", length(removed_leakage), "leakage variables:\n")
  cat(paste(" -", removed_leakage, collapse="\n"), "\n\n")
}

## Removing 20 leakage variables:
##  - recoveries
##  - collection_recovery_fee
##  - last_pymnt_amnt
##  - last_pymnt_d_since_issue
##  - out_prncp
##  - out_prncp_inv
##  - total_pymnt
##  - total_pymnt_inv
##  - total_rec_prncp
##  - total_rec_int
##  - total_rec_late_fee
##  - last_credit_pull_d_since_issue
##  - last_fico_range_high
```

```
##  - last_fico_range_low
##  - hardship_flag
##  - has_hardship
##  - ever_hardship
##  - pymnt_plan
##  - has_payment_plan
##  - debt_settlement_flag
```

```r
X <- credit[, !drop_cols, with=FALSE]
Y <- credit$loan_status_bin

# data_raw: original values
data_raw <- copy(X)
invisible(data_raw[, loan_status_bin := Y])

# data_scaled: standardized, for Logistic Regression only
data_scaled <- copy(X)
num_cols    <- names(data_scaled)[sapply(data_scaled, is.numeric)]
for (col in num_cols) {
  mu <- mean(data_scaled[[col]], na.rm=TRUE)
  sg <- sd(data_scaled[[col]],   na.rm=TRUE)
  if (sg > 0) data_scaled[, (col) := (get(col) - mu) / sg]
}
invisible(data_scaled[, loan_status_bin := Y])
```

4. Information Value (IV) & Variable Selection

```r
dt_iv <- copy(data_raw)
invisible(dt_iv[, y := fifelse(loan_status_bin=="Bad", 1L, 0L)])
invisible(dt_iv[, loan_status_bin := NULL])

iv_list     <- iv(dt_iv, y="y")
iv_selected <- iv_list[iv_list$info_value >= 0.02, ]
iv_selected <- iv_selected[order(-iv_selected$info_value), ]
selected_vars <- iv_selected$variable

cat("Top 10 variables by IV:\n"); print(head(iv_selected, 10))
```

```
## Top 10 variables by IV:

##                       variable info_value
##                         <char>      <num>
##  1:                installment  1.0314910
##  2:                   int_rate  0.8410529
##  3:             revol_bal_used  0.6006864
##  4:             bc_open_to_buy  0.5957694
##  5:                avg_cur_bal  0.5945951
##  6:                income_used  0.5796189
##  7:                  sub_grade  0.5443808
```

```
##  8:                       grade  0.5020304
##  9: total_il_high_credit_limit  0.4980451
## 10:          total_bal_ex_mort  0.3594594
```

```r
cat("\nNumber of selected variables:", length(selected_vars), "\n")
```

```
##
## Number of selected variables: 44
```

5. Train / Test Split (70/30, Stratified)

```r
set.seed(SEED)
train_idx <- createDataPartition(Y, p=0.7, list=FALSE)

train_raw <- data_raw[train_idx, ];  test_raw  <- data_raw[-train_idx, ]
invisible(train_raw[, y := fifelse(loan_status_bin=="Bad", 1L, 0L)])
invisible(test_raw[,  y := fifelse(loan_status_bin=="Bad", 1L, 0L)])

train_scaled <- data_scaled[train_idx, ]; test_scaled <- data_scaled[-
train_idx, ]

cat("Train:", nrow(train_raw), "| Test:", nrow(test_raw), "\n")
```

```
## Train: 140001 | Test: 59999
```

```r
cat("Bad rate - Train:", round(mean(train_raw$loan_status_bin=="Bad")*100,2),
"%\n")
```

```
## Bad rate - Train: 13.04 %
```

```r
cat("Bad rate - Test :", round(mean(test_raw$loan_status_bin=="Bad")*100,2),
"%\n")
```

```
## Bad rate - Test : 13.04 %
```

6. Model Evaluation Helper

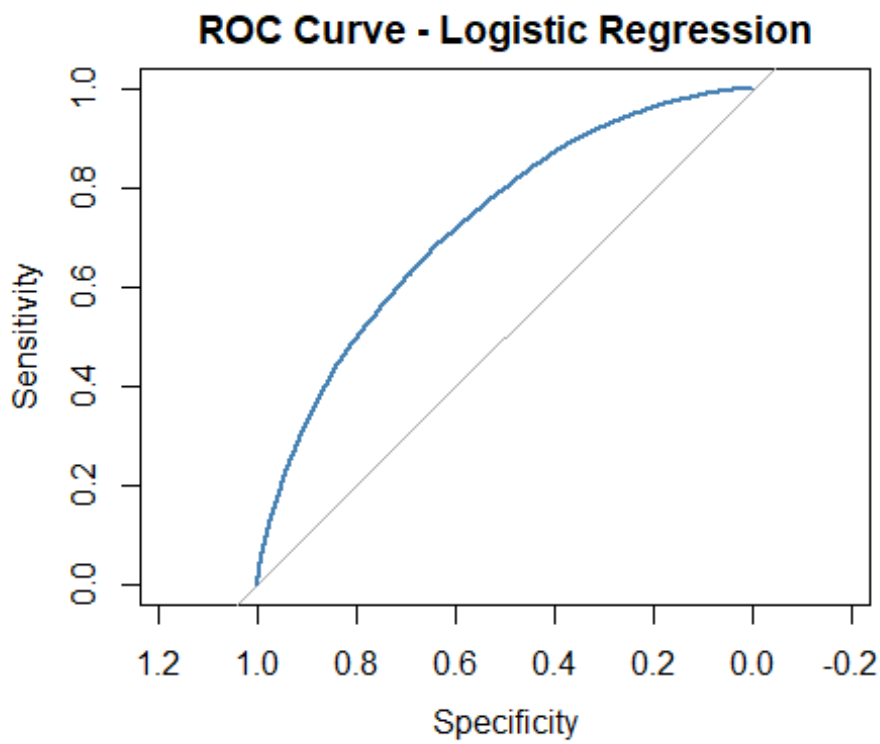
```r
eval_model <- function(actual, pred_prob, threshold=0.5,
                       model_name="Model", train_time=NA, predict_time=NA) {
  pred_class <- factor(ifelse(pred_prob >= threshold,"Bad","Good"),
levels=c("Good","Bad"))
  cat("\n========", model_name, "========\n")
  if (!is.na(train_time))   cat(sprintf("Train time   : %.2f sec\n",
train_time))
  if (!is.na(predict_time)) cat(sprintf("Predict time : %.2f sec\n",
predict_time))
  print(confusionMatrix(pred_class, actual, positive="Bad"))
  roc_obj  <- roc(actual, pred_prob, levels=c("Good","Bad"), direction="<",
quiet=TRUE)
  auc_val  <- auc(roc_obj); gini_val <- 2*auc_val - 1
```

```
  cat("AUC :", round(auc_val,4), "\nGini:", round(gini_val,4), "\n")
  plot(roc_obj, col="steelblue", lwd=2, main=paste("ROC Curve -",
model_name))
  invisible(list(roc=roc_obj, auc=auc_val, gini=gini_val,
                 train_time=train_time, predict_time=predict_time))
}
```

7. Logistic Regression (using data_scaled)

```
formula_lr <- as.formula(paste("loan_status_bin ~", paste(selected_vars,
collapse=" + ")))

t0 <- proc.time()
logit_model <- glm(formula_lr, data=train_scaled, family=binomial)
time_logit_train <- (proc.time()-t0)[["elapsed"]]

t0 <- proc.time()
pred_prob_logit <- predict(logit_model, newdata=test_scaled, type="response")
time_logit_pred <- (proc.time()-t0)[["elapsed"]]

results_logit <- eval_model(test_scaled$loan_status_bin, pred_prob_logit,
                            model_name="Logistic Regression",
                            train_time=time_logit_train,
predict_time=time_logit_pred)

##
## ======== Logistic Regression ========
## Train time   : 4.61 sec
## Predict time : 0.34 sec
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  Good   Bad
##       Good 51774  7384
##       Bad    404   437
##
##                Accuracy : 0.8702
##                  95% CI : (0.8675, 0.8729)
##     No Information Rate : 0.8696
##     P-Value [Acc > NIR] : 0.3472
##
##                   Kappa : 0.0776
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.055875
##             Specificity : 0.992257
##          Pos Pred Value : 0.519620
##          Neg Pred Value : 0.875182
```

```
##              Prevalence : 0.130352
##          Detection Rate : 0.007283
##    Detection Prevalence : 0.014017
##       Balanced Accuracy : 0.524066
##
##        'Positive' Class : Bad
##
## AUC : 0.7248
## Gini: 0.4497
```

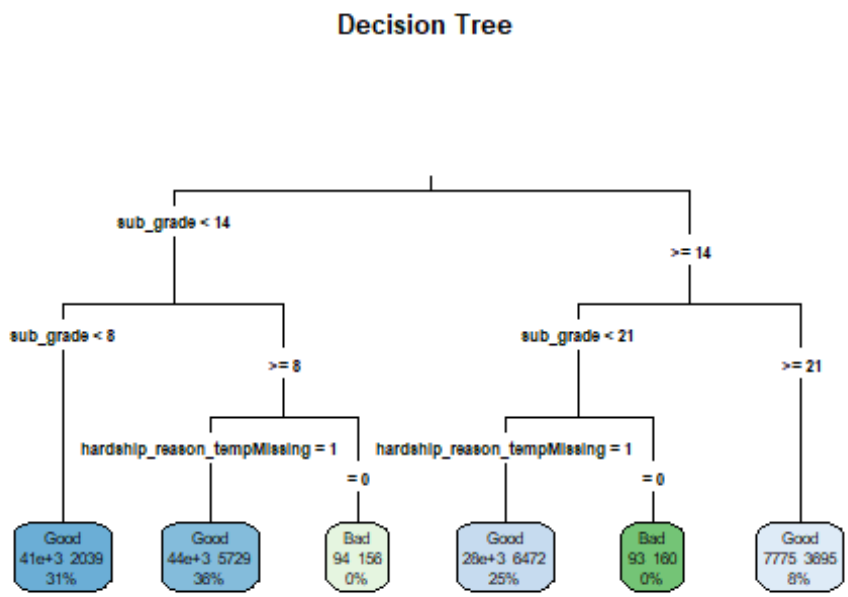


8. Decision Tree (using data_raw)

cp = 0.001 (reduced from 0.01) to allow deeper splits and avoid underfitting. minsplit = 100 ensures each node has sufficient observations before splitting. maxdepth = 6 caps tree depth to prevent overfitting.

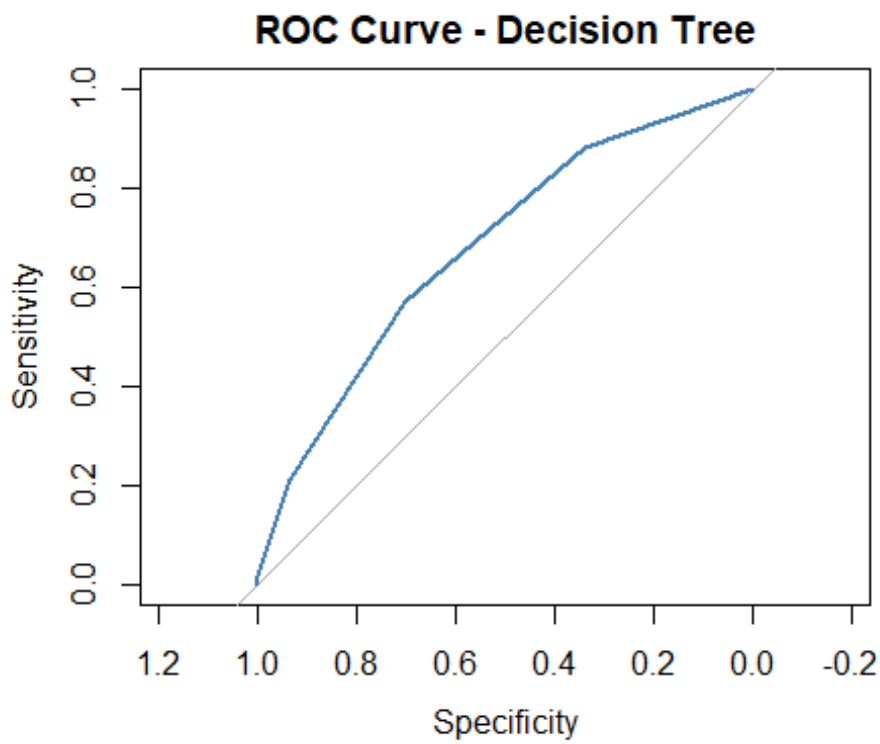
```
formula_dt <- as.formula(paste("loan_status_bin ~", paste(selected_vars,
collapse=" + ")))

t0 <- proc.time()
dt_model <- rpart(formula_dt, data=train_raw, method="class",
                  control=rpart.control(cp=0.001, minsplit=100, maxdepth=6))
time_dt_train <- (proc.time()-t0)[["elapsed"]]
```

```r
rpart.plot(dt_model, type=3, extra=101, fallen.leaves=TRUE, main="Decision
Tree")
```




```r
t0 <- proc.time()
pred_prob_dt <- predict(dt_model, newdata=test_raw, type="prob")[,"Bad"]
time_dt_pred <- (proc.time()-t0)[["elapsed"]]

results_dt <- eval_model(test_raw$loan_status_bin, pred_prob_dt,
                         model_name="Decision Tree",
                         train_time=time_dt_train, predict_time=time_dt_pred)

##
## ======== Decision Tree ========
## Train time   : 13.18 sec
## Predict time : 0.11 sec
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  Good   Bad
##       Good 52087  7702
##       Bad     91   119
##
##                Accuracy : 0.8701
##                  95% CI : (0.8674, 0.8728)
##     No Information Rate : 0.8696
##     P-Value [Acc > NIR] : 0.3699
##
```

```
##                   Kappa : 0.023
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.015215
##             Specificity : 0.998256
##          Pos Pred Value : 0.566667
##          Neg Pred Value : 0.871180
##              Prevalence : 0.130352
##          Detection Rate : 0.001983
##    Detection Prevalence : 0.003500
##       Balanced Accuracy : 0.506736
##
##        'Positive' Class : Bad
##
## AUC : 0.6798
## Gini: 0.3595
```



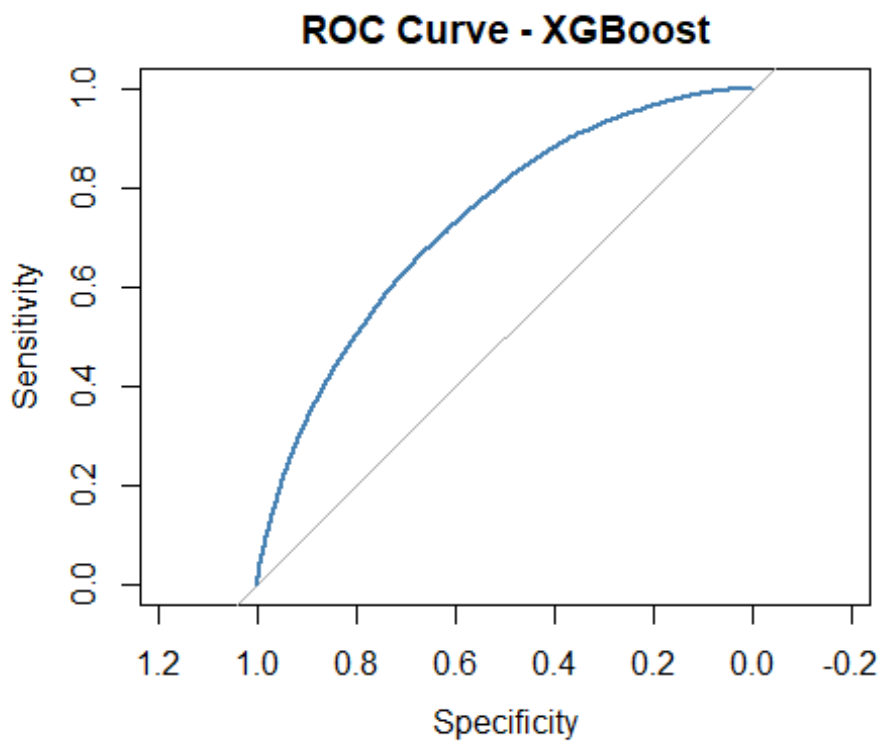
9. XGBoost (using data_raw)

```
train_matrix <- xgb.DMatrix(data=as.matrix(train_raw[, selected_vars,
with=FALSE]),
                            label=train_raw$y)
test_matrix  <- xgb.DMatrix(data=as.matrix(test_raw[,  selected_vars,
with=FALSE]),
                            label=test_raw$y)
```

```r
xgb_params <- list(objective="binary:logistic", eval_metric="auc",
                   max_depth=6, eta=0.1, subsample=0.8, colsample_bytree=0.8)

t0 <- proc.time()
xgb_model <- xgb.train(params=xgb_params, data=train_matrix, nrounds=200,
                       watchlist=list(train=train_matrix, eval=test_matrix),
                       early_stopping_rounds=20, verbose=0)
time_xgb_train <- (proc.time()-t0)[["elapsed"]]

t0 <- proc.time()
invisible(test_raw[, pred_prob_xgb := predict(xgb_model,
newdata=test_matrix)])
time_xgb_pred <- (proc.time()-t0)[["elapsed"]]

results_xgb <- eval_model(test_raw$loan_status_bin, test_raw$pred_prob_xgb,
                          model_name="XGBoost",
                          train_time=time_xgb_train,
predict_time=time_xgb_pred)
```

```
##
## ======== XGBoost ========
## Train time   : 3.64 sec
## Predict time : 0.05 sec
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  Good   Bad
##       Good 51907  7525
##       Bad    271   296
##
##                Accuracy : 0.8701
##                  95% CI : (0.8673, 0.8727)
##     No Information Rate : 0.8696
##     P-Value [Acc > NIR] : 0.3837
##
##                   Kappa : 0.0539
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.037847
##             Specificity : 0.994806
##          Pos Pred Value : 0.522046
##          Neg Pred Value : 0.873385
##              Prevalence : 0.130352
##          Detection Rate : 0.004933
##    Detection Prevalence : 0.009450
##       Balanced Accuracy : 0.516327
##
```

```
##        'Positive' Class : Bad
##
## AUC : 0.7323
## Gini: 0.4647
```

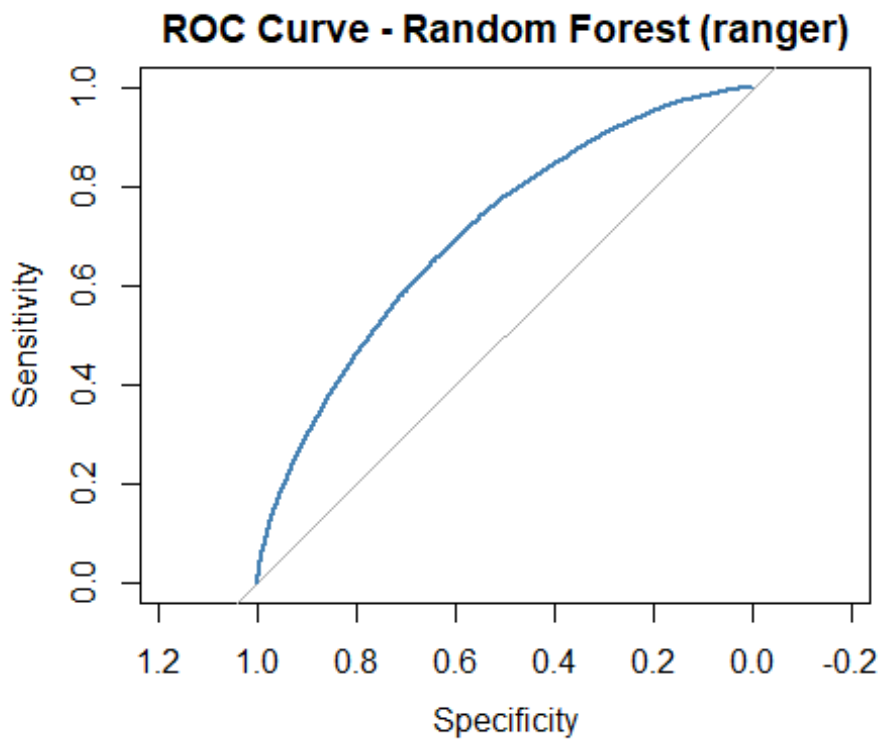


10. Random Forest - ranger (using data_raw)

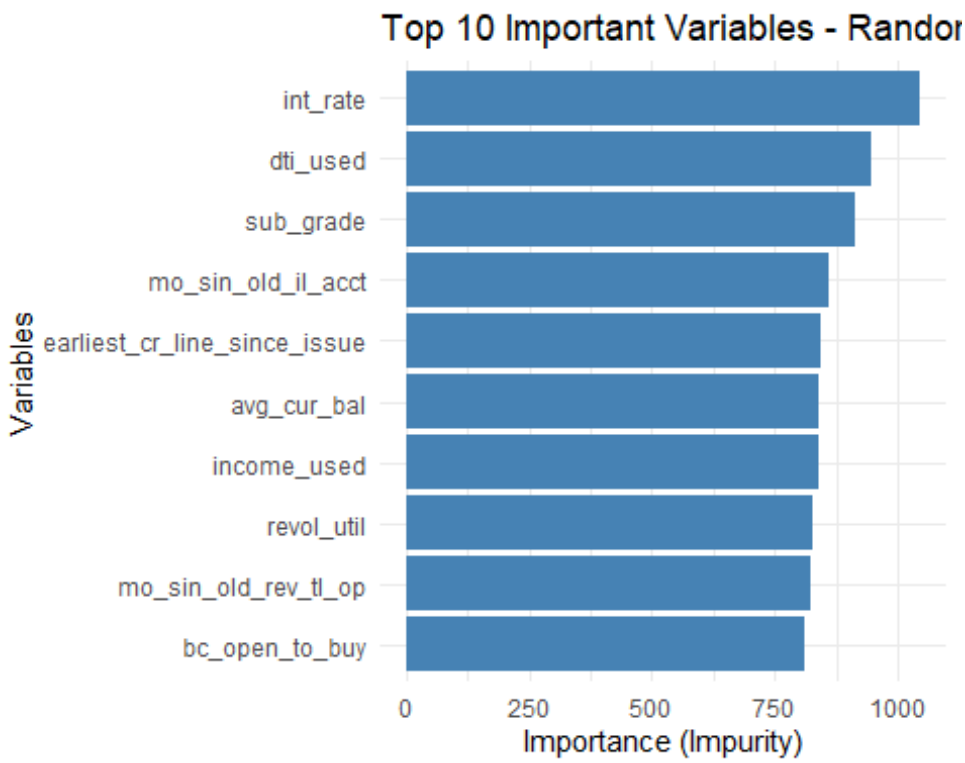
```
rf_formula  <- as.formula(paste("loan_status_bin ~", paste(selected_vars,
collapse=" + ")))
rf_train_df <- as.data.frame(train_raw[, c(selected_vars, "loan_status_bin"),
with=FALSE])
rf_test_df  <- as.data.frame(test_raw[,  selected_vars, with=FALSE])

t0 <- proc.time()
rf_model <- ranger(formula    = rf_formula,
                   data       = rf_train_df,
                   num.trees  = 100,
                   mtry       = floor(sqrt(length(selected_vars))),
                   importance = "impurity",
                   probability= TRUE,
                   seed       = SEED)
time_rf_train <- (proc.time()-t0)[["elapsed"]]

t0 <- proc.time()
invisible(test_raw[, pred_prob_rf := predict(rf_model,
data=rf_test_df)$predictions[, "Bad"]])
```

```r
time_rf_pred <- (proc.time()-t0)[["elapsed"]]

results_rf <- eval_model(test_raw$loan_status_bin, test_raw$pred_prob_rf,
                         model_name="Random Forest (ranger)",
                         train_time=time_rf_train, predict_time=time_rf_pred)
```

```
##
## ======== Random Forest (ranger) ========
## Train time   : 29.62 sec
## Predict time : 1.97 sec
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  Good   Bad
##       Good 52036  7632
##       Bad    142   189
##
##                Accuracy : 0.8704
##                  95% CI : (0.8677, 0.8731)
##     No Information Rate : 0.8696
##     P-Value [Acc > NIR] : 0.2868
##
##                   Kappa : 0.0362
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.024166
##             Specificity : 0.997279
##          Pos Pred Value : 0.570997
##          Neg Pred Value : 0.872092
##              Prevalence : 0.130352
##          Detection Rate : 0.003150
##    Detection Prevalence : 0.005517
##       Balanced Accuracy : 0.510722
##
##        'Positive' Class : Bad
##
## AUC : 0.7049
## Gini: 0.4097
```




```
importance_df <- data.frame(Variable   = names(rf_model$variable.importance),
                            Importance = rf_model$variable.importance)
importance_df <- importance_df[order(-importance_df$Importance), ]
top3_vars     <- head(importance_df$Variable, 3)

ggplot(head(importance_df, 10), aes(x=reorder(Variable, Importance),
y=Importance)) +
  geom_bar(stat="identity", fill="steelblue") + coord_flip() +
  labs(title="Top 10 Important Variables - Random Forest",
       x="Variables", y="Importance (Impurity)") + theme_minimal()
```

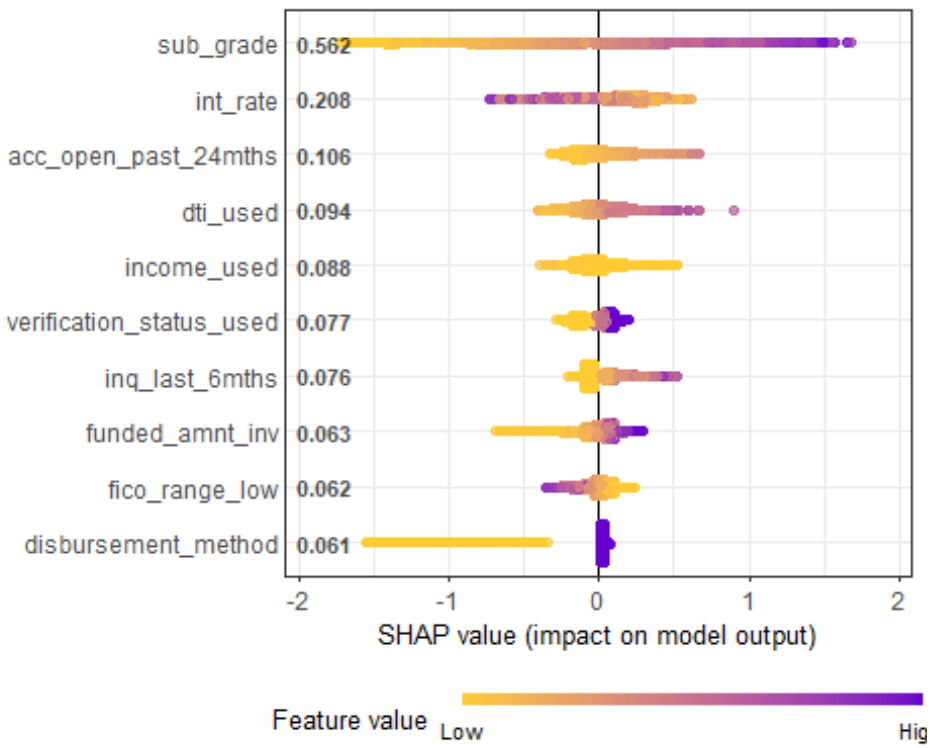


11. Model Comparison

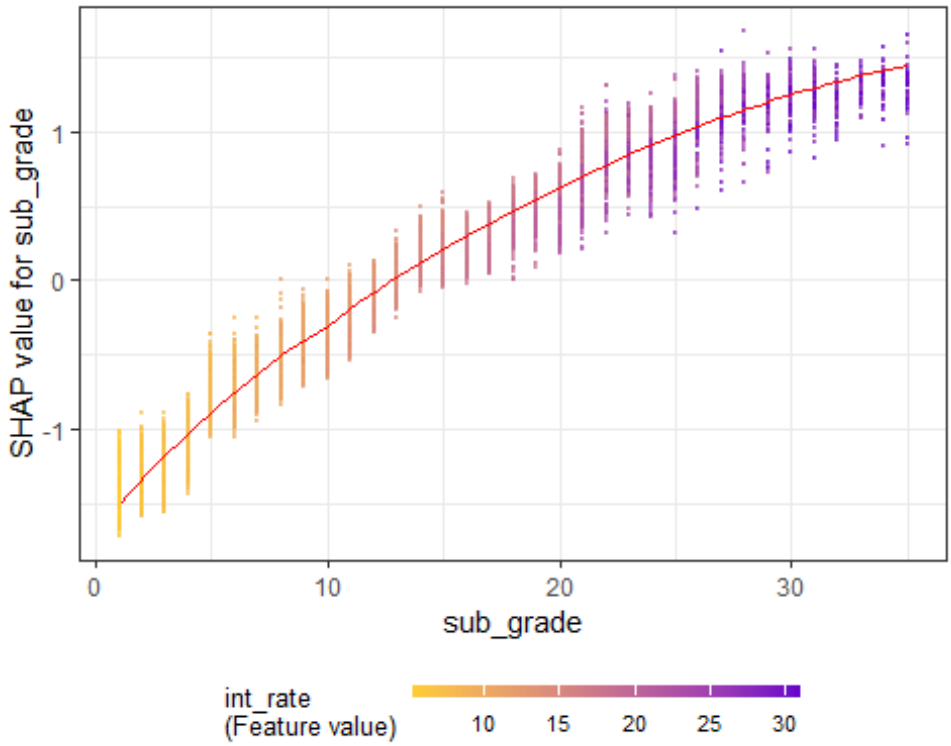
```
comparison <- data.frame(
  Model       = c("Logistic Regression","Decision Tree","XGBoost","Random
Forest"),
  AUC         = round(c(results_logit$auc,  results_dt$auc,
                        results_xgb$auc,    results_rf$auc), 4),
  Gini        = round(c(results_logit$gini, results_dt$gini,
                        results_xgb$gini,   results_rf$gini), 4),
  Train_sec   = round(c(results_logit$train_time,   results_dt$train_time,
                        results_xgb$train_time,     results_rf$train_time),
2),
  Predict_sec = round(c(results_logit$predict_time, results_dt$predict_time,
                        results_xgb$predict_time,   results_rf$predict_time),
2)
)
cat("========== MODEL COMPARISON TABLE ==========\n"); print(comparison)

## ========== MODEL COMPARISON TABLE ==========

##                  Model    AUC   Gini Train_sec Predict_sec
## 1 Logistic Regression 0.7248 0.4497      4.61        0.34
## 2       Decision Tree 0.6798 0.3595     13.18        0.11
## 3             XGBoost 0.7323 0.4647      3.64        0.05
## 4       Random Forest 0.7049 0.4097     29.62        1.97
```

12. SHAP – Model Interpretation

```
X_test_matrix <- as.matrix(test_raw[, selected_vars, with=FALSE])
shap_values   <- shap.values(xgb_model=xgb_model, X_train=X_test_matrix)
shap_score    <- shap_values$shap_score
shap_long     <- shap.prep(xgb_model=xgb_model, X_train=X_test_matrix,
top_n=10)
top_var       <- names(sort(colMeans(abs(shap_score)), decreasing=TRUE))[1]
cat("Most important SHAP variable:", top_var, "\n")

## Most important SHAP variable: sub_grade

shap.plot.summary(shap_long, dilute=TRUE)
```




```
shap.plot.dependence(data_long=shap_long, x=top_var, color_feature="auto")
```

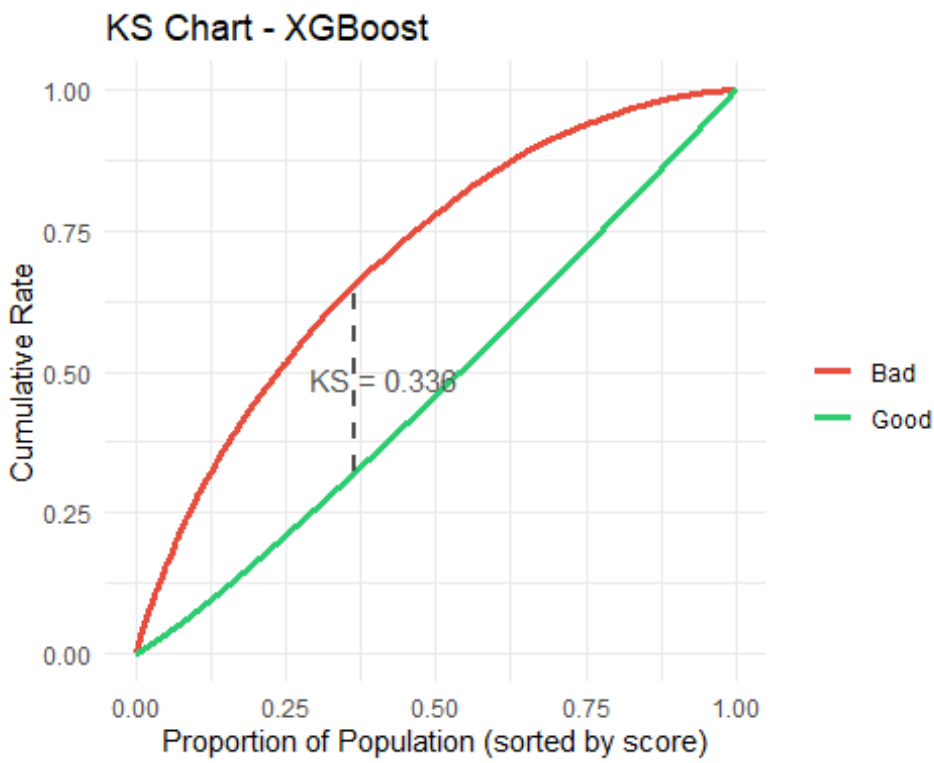


13. KS / PSI – Model Validation

13.1 KS Statistic (threshold >= 0.3)

```
compute_ks <- function(actual, pred_prob, model_name="Model") {
  y  <- ifelse(actual=="Bad", 1, 0)
  df <- data.frame(y=y, prob=pred_prob)[order(-pred_prob), ]
  df$cum_bad  <- cumsum(df$y==1) / sum(y==1)
  df$cum_good <- cumsum(df$y==0) / sum(y==0)
  df$ks       <- abs(df$cum_bad - df$cum_good)
  ks_val      <- max(df$ks)
  cat(sprintf("%-25s KS = %.4f  (%s)\n", model_name, ks_val,
              ifelse(ks_val>=0.3,"PASS","FAIL")))
  invisible(list(ks=ks_val, df=df))
}

cat("========== KS STATISTIC ==========\n")

## ========== KS STATISTIC ==========

ks_logit <- compute_ks(test_scaled$loan_status_bin, pred_prob_logit,
"Logistic Regression")

## Logistic Regression       KS = 0.3229  (PASS)
```

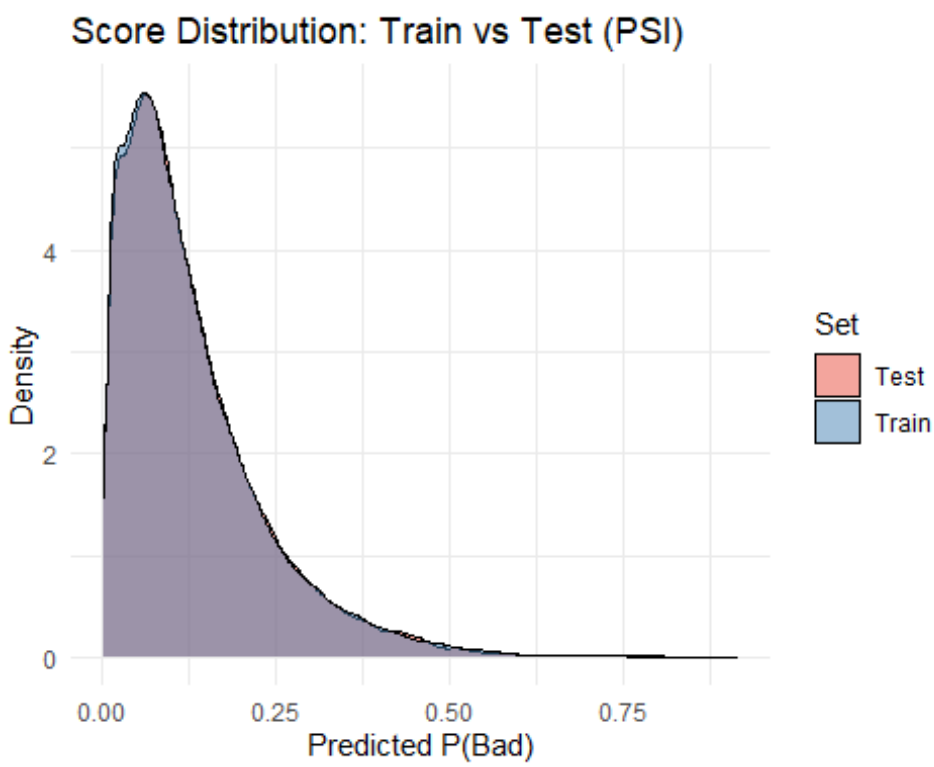
```r
ks_dt    <- compute_ks(test_raw$loan_status_bin,    pred_prob_dt,
"Decision Tree")

## Decision Tree             KS = 0.2709  (FAIL)

ks_xgb   <- compute_ks(test_raw$loan_status_bin,    test_raw$pred_prob_xgb,
"XGBoost")

## XGBoost                   KS = 0.3358  (PASS)

ks_rf    <- compute_ks(test_raw$loan_status_bin,    test_raw$pred_prob_rf,
"Random Forest")

## Random Forest             KS = 0.2940  (FAIL)

ks_df <- ks_xgb$df; ks_df$decile <- seq(0, 1, length.out=nrow(ks_df))
ks_max_idx <- which.max(ks_df$ks)

ggplot(ks_df, aes(x=decile)) +
  geom_line(aes(y=cum_bad,  color="Bad"),  linewidth=1.2) +
  geom_line(aes(y=cum_good, color="Good"), linewidth=1.2) +
  geom_segment(x=ks_df$decile[ks_max_idx], xend=ks_df$decile[ks_max_idx],
               y=ks_df$cum_good[ks_max_idx], yend=ks_df$cum_bad[ks_max_idx],
               color="gray30", linetype="dashed", linewidth=0.8) +
  annotate("text", x=ks_df$decile[ks_max_idx]+0.05,
           y=(ks_df$cum_bad[ks_max_idx]+ks_df$cum_good[ks_max_idx])/2,
           label=sprintf("KS = %.3f", ks_xgb$ks), color="gray30", size=4) +
  scale_color_manual(values=c("Bad"="#e74c3c","Good"="#2ecc71")) +
  labs(title="KS Chart - XGBoost",
       x="Proportion of Population (sorted by score)", y="Cumulative Rate",
color="") +
  theme_minimal()
```



13.2 PSI (< 0.1: Stable | 0.1-0.2: Monitor | > 0.2: Drift)

```
compute_psi <- function(expected, actual, n_bins=10) {
  breaks  <- unique(quantile(expected, probs=seq(0,1,1/n_bins), na.rm=TRUE))
  exp_pct <- pmax(as.numeric(table(cut(expected, breaks,
include.lowest=TRUE)))/length(expected), 1e-6)
  act_pct <- pmax(as.numeric(table(cut(actual,   breaks,
include.lowest=TRUE)))/length(actual),   1e-6)
  psi_val <- sum((act_pct - exp_pct) * log(act_pct/exp_pct))
  cat(sprintf("PSI = %.4f -> %s\n", psi_val,
              ifelse(psi_val<0.1,"Stable",
ifelse(psi_val<0.2,"Monitor","Model Drift"))))
  invisible(psi_val)
}

pred_prob_train_xgb <- predict(xgb_model,
  newdata=xgb.DMatrix(as.matrix(train_raw[, selected_vars, with=FALSE])))

cat("========== PSI - XGBoost ==========\n")

## ========== PSI - XGBoost ==========

psi_xgb <- compute_psi(pred_prob_train_xgb, test_raw$pred_prob_xgb)

## PSI = 0.0003 -> Stable
```

```r
data.frame(Score = c(pred_prob_train_xgb, test_raw$pred_prob_xgb),
           Set   = c(rep("Train", length(pred_prob_train_xgb)),
                     rep("Test",  nrow(test_raw)))) |>
  ggplot(aes(x=Score, fill=Set)) + geom_density(alpha=0.5) +
  scale_fill_manual(values=c("Train"="steelblue","Test"="#e74c3c")) +
  labs(title="Score Distribution: Train vs Test (PSI)",
       x="Predicted P(Bad)", y="Density") +
  theme_minimal()
```



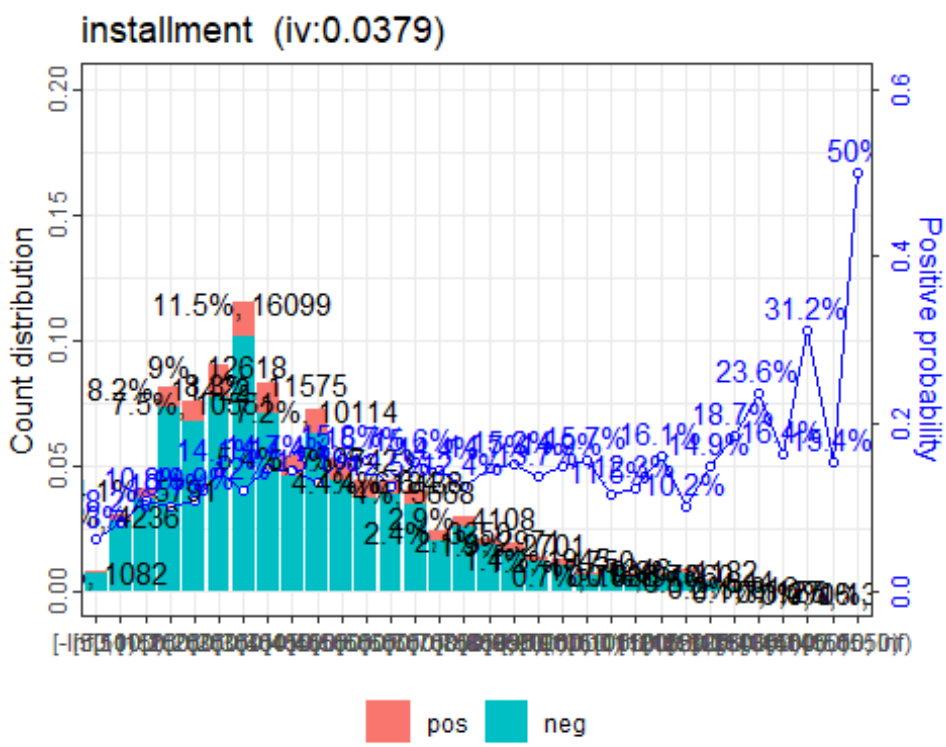
14. Scorecard – Credit Scoring (using data_raw)

3 optimizations to reduce RAM usage:

- Use train_raw instead of copying the full credit dataset
- Limit to top 10 IV variables (sufficient for a scorecard model)
- Switch method from “tree” to “chimerge” (lighter computation)

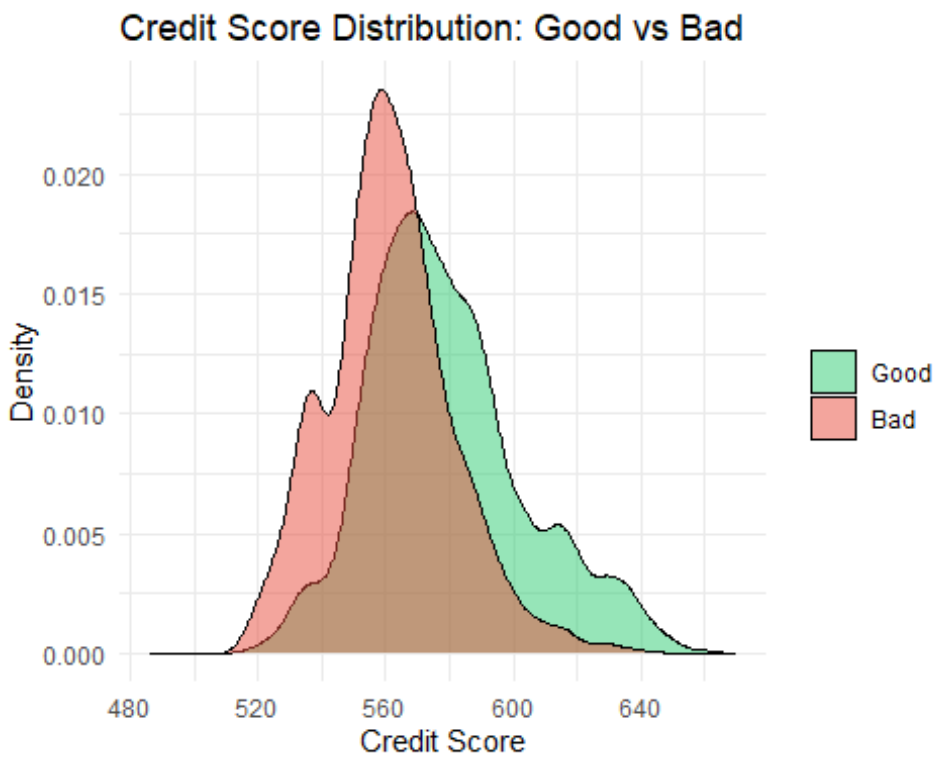
```r
sc_data <- as.data.frame(train_raw)
sc_data$y <- as.integer(sc_data$loan_status_bin == "Bad")
sc_data$loan_status_bin <- NULL

sc_vars <- head(iv_selected$variable, 10)
sc_vars <- intersect(sc_vars, names(sc_data))

# Remove binary variables (only 2 unique values) -- woebin cannot bin them
```

```r
correctly
binary_vars <- sc_vars[sapply(sc_vars, function(v)
length(unique(sc_data[[v]])) <= 2)]
if (length(binary_vars) > 0) {
  cat("Removing binary variables:", paste(binary_vars, collapse=", "), "\n")
  sc_vars <- setdiff(sc_vars, binary_vars)
}

sc_data <- sc_data[, c(sc_vars, "y")]
bins <- woebin(sc_data, y="y", method="chimerge")

## ✓ Binning on 140001 rows and 11 columns in 00:00:11

woebin_plot(bins[sc_vars[1]])

## $installment
```

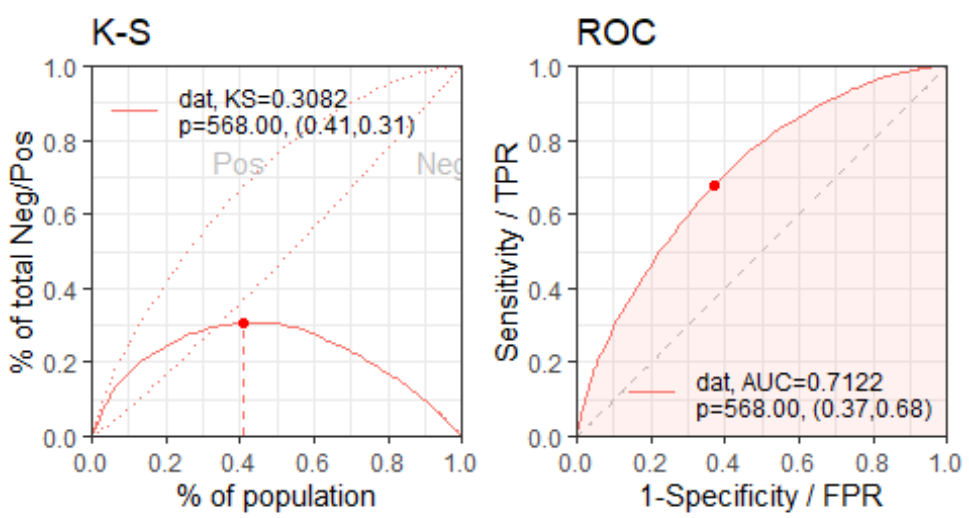



```r
# Split raw sc_data first -> scorecard_ply needs original (non-WOE) values
set.seed(SEED)
sc_idx <- createDataPartition(sc_data$y, p=0.7, list=FALSE)

sc_raw_train <- sc_data[sc_idx,  sc_vars, drop=FALSE]
sc_raw_test  <- sc_data[-sc_idx, sc_vars, drop=FALSE]

sc_train_woe   <- woebin_ply(sc_raw_train, bins)

## ✓ Woe transformating on 98001 rows and 10 columns in 00:00:11
```

```r
sc_train_woe$y <- sc_data[sc_idx, "y"]

sc_model <- glm(y ~ ., data=sc_train_woe, family=binomial)

# PDO=20: odds double every 20 points | base score 600 at odds 1/19
card <- scorecard(bins, sc_model, points0=600, odds0=1/19, pdo=20)

score_train <- scorecard_ply(sc_raw_train, card)[[1]]
score_test  <- scorecard_ply(sc_raw_test,  card)[[1]]

cat("NA train:", sum(is.na(score_train)),
    "| NA test:", sum(is.na(score_test)), "\n")

## NA train: 0 | NA test: 0

sc_raw_train$score <- score_train
sc_raw_test$score  <- score_test
sc_raw_train$y     <- sc_data[sc_idx,  "y"]
sc_raw_test$y      <- sc_data[-sc_idx, "y"]

cat(sprintf("Score Min : %.0f\nScore Mean: %.0f\nScore Max : %.0f\n",
            min(score_test,  na.rm=TRUE),
            mean(score_test, na.rm=TRUE),
            max(score_test,  na.rm=TRUE)))

## Score Min : 486
## Score Mean: 577
## Score Max : 670

ggplot(sc_raw_test, aes(x=score, fill=factor(y))) +
  geom_density(alpha=0.5) +
  scale_fill_manual(values=c("0"="#2ecc71","1"="#e74c3c"),
labels=c("Good","Bad")) +
  labs(title="Credit Score Distribution: Good vs Bad",
       x="Credit Score", y="Density", fill="") +
  theme_minimal()
```




```
perf_eva(pred=sc_raw_test$score, label=sc_raw_test$y,
show_plot=c("ks","roc"))
```



```
## $binomial_metric
## $binomial_metric$dat
##           KS       AUC      Gini
##        <num>     <num>     <num>
## 1: 0.3082231 0.7121697 0.4243394
##
##
## $pic
## TableGrob (1 x 2) "arrange": 2 grobs
##   z     cells    name           grob
## 1 1 (1-1,1-1) arrange gtable[layout]
## 2 2 (1-1,2-2) arrange gtable[layout]
```

15. Expected Loss (EL = PD x LGD x EAD)

Two-stage pipeline:

- **Stage 1 (Modeling)**: Use information available at loan origination -> model outputs PD
- **Stage 2 (Risk Management)**: Combine PD with LGD (Basel II constant) and EAD (loan_amnt) -> compute EL

All evaluation is performed on the **held-out test set only** – the model has never seen these observations during training, ensuring unbiased loss estimates.

15.1 Stage 1: Predict PD using XGBoost (Test Set Only)

`test_matrix` was already created in chunk `xgboost` – reused here directly.

```
PD <- predict(xgb_model, newdata=test_matrix)

cat(sprintf("Test set size : %s loans\n", format(nrow(test_raw),
big.mark=",")))

## Test set size : 59,999 loans

cat(sprintf("Mean PD       : %.2f%%\n",   mean(PD) * 100))

## Mean PD       : 12.99%

cat(sprintf("PD min / max  : %.2f%% / %.2f%%\n", min(PD)*100, max(PD)*100))

## PD min / max  : 0.14% / 88.82%
```

15.2 Stage 2: Compute EL = PD x LGD x EAD

LGD = 45% per Basel II for unsecured loans – a historical constant, not derived from data.
EAD = loan_amnt – known at origination.

```r
el_data <- data.frame(
  EAD    = test_raw$loan_amnt,                                  # Exposure at
Default
  PD     = PD,                                                  # From XGBoost
(Stage 1)
  LGD    = LGD_ASSUMED,                                         # 45% Basel II
constant
  actual = ifelse(test_raw$loan_status_bin == "Bad", 1, 0)   # For back-test
only
)

el_data$EL <- el_data$PD     * el_data$LGD * el_data$EAD  # Predicted EL
el_data$AL <- el_data$actual * el_data$LGD * el_data$EAD  # Actual Loss

cat("========== EXPECTED LOSS SUMMARY (TEST SET) ==========\n")

## ========== EXPECTED LOSS SUMMARY (TEST SET) ==========

cat(sprintf("Evaluated on            : Test set (%s loans)\n",
format(nrow(el_data), big.mark=",")))

## Evaluated on            : Test set (59,999 loans)

cat(sprintf("LGD assumed (Basel II)  : %.0f%%\n",  LGD_ASSUMED * 100))

## LGD assumed (Basel II)  : 45%

cat(sprintf("Total EAD               : $%s\n",
format(round(sum(el_data$EAD)), big.mark=",")))

## Total EAD               : $919,275,525

cat(sprintf("Total EL (predicted)    : $%s\n",
format(round(sum(el_data$EL)),  big.mark=",")))

## Total EL (predicted)    : $56,399,097

cat(sprintf("Total Actual Loss       : $%s\n",
format(round(sum(el_data$AL)),  big.mark=",")))

## Total Actual Loss       : $56,539,980

cat(sprintf("EL / EAD                : %.2f%%\n",   sum(el_data$EL) /
sum(el_data$EAD) * 100))

## EL / EAD                : 6.14%

cat(sprintf("AL / EAD (actual)       : %.2f%%\n",   sum(el_data$AL) /
sum(el_data$EAD) * 100))

## AL / EAD (actual)       : 6.15%
```

```r
cat(sprintf("Difference (EL - AL)    : $%s (%.2f%%)\n",
            format(round(sum(el_data$EL) - sum(el_data$AL)), big.mark=","),
            (sum(el_data$EL) - sum(el_data$AL)) / sum(el_data$AL) * 100))
```

```
## Difference (EL - AL)    : $-140,883 (-0.25%)
```

15.3 EL Analysis by PD Risk Group

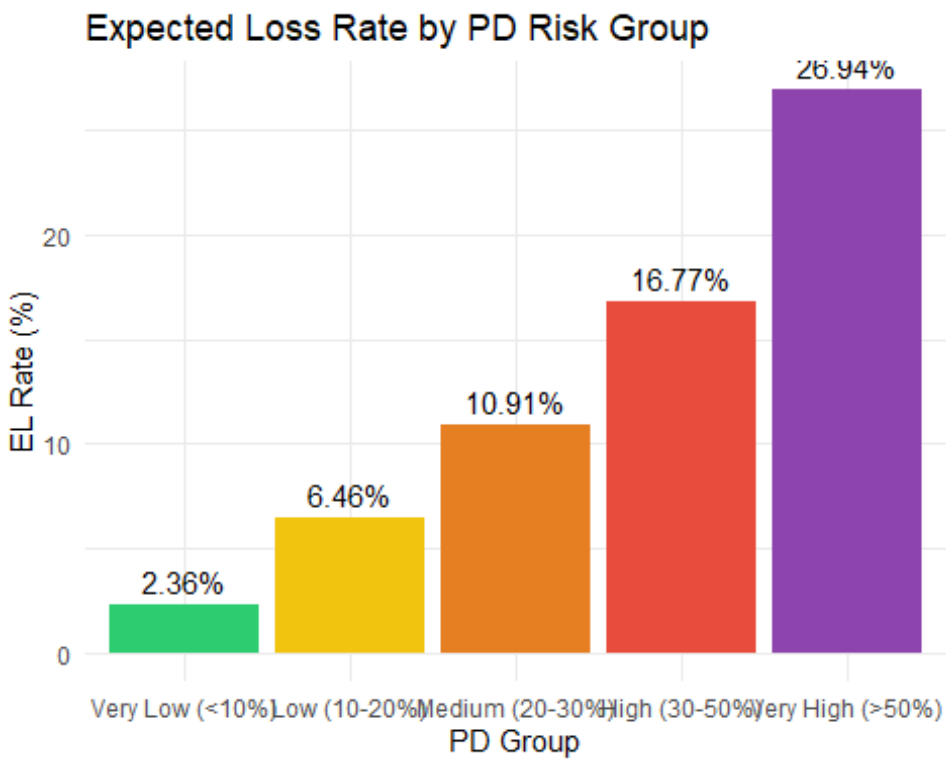
```r
el_data$pd_group <- cut(el_data$PD,
                        breaks = c(0, 0.1, 0.2, 0.3, 0.5, 1),
                        labels = c("Very Low (<10%)", "Low (10-20%)",
                                   "Medium (20-30%)", "High (30-50%)",
                                   "Very High (>50%)"),
                        include.lowest = TRUE)

el_summary <- el_data |>
  group_by(pd_group) |>
  summarise(
    n_loans   = n(),
    total_EAD = round(sum(EAD)),
    total_EL  = round(sum(EL)),
    EL_rate   = round(sum(EL) / sum(EAD) * 100, 2),
    avg_PD    = round(mean(PD) * 100, 2),
    .groups   = "drop"
  )

cat("========== EXPECTED LOSS BY RISK GROUP ==========\n")
```

```
## ========== EXPECTED LOSS BY RISK GROUP ==========
```

```r
print(el_summary)
```

```
## # A tibble: 5 × 6
##   pd_group         n_loans total_EAD total_EL EL_rate avg_PD
##   <fct>              <int>     <dbl>    <dbl>   <dbl>  <dbl>
## 1 Very Low (<10%)    29544 437666350 10341007    2.36   5.29
## 2 Low (10-20%)       18664 279993000 18075563    6.46  14.3
## 3 Medium (20-30%)     7293 118525375 12931928   10.9   24.2
## 4 High (30-50%)       3931  72151450 12103171   16.8   37.1
## 5 Very High (>50%)     567  10939350  2947430   26.9   59.8
```

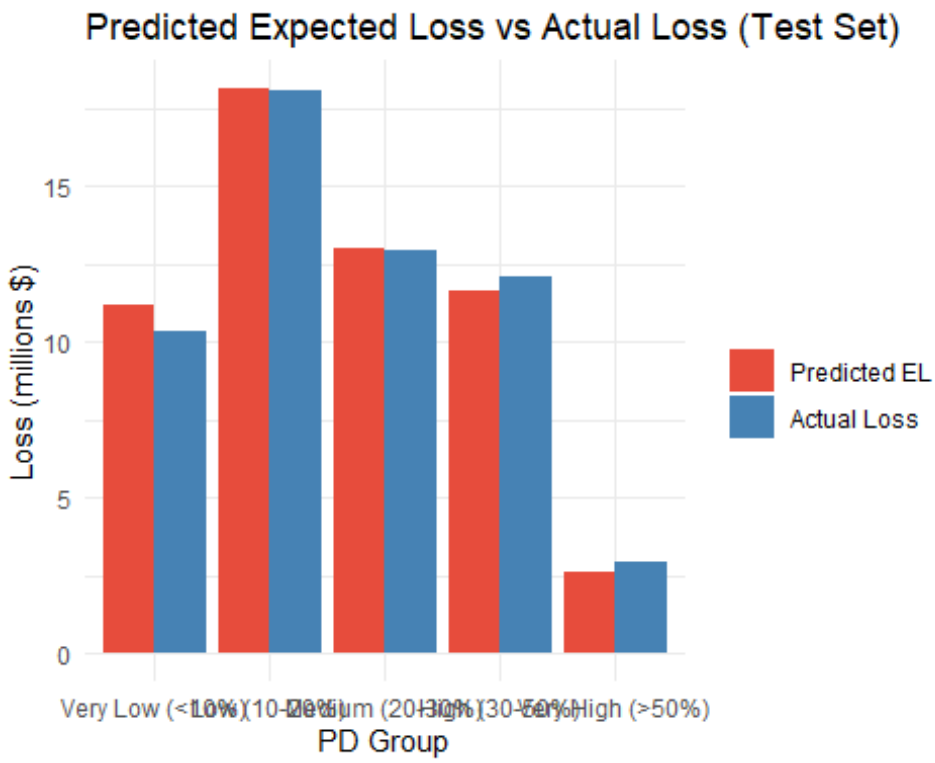
```r
ggplot(el_summary, aes(x=pd_group, y=EL_rate, fill=pd_group)) +
  geom_bar(stat="identity", show.legend=FALSE) +
  geom_text(aes(label=paste0(EL_rate, "%")), vjust=-0.5, size=4) +

scale_fill_manual(values=c("#2ecc71","#f1c40f","#e67e22","#e74c3c","#8e44ad")
) +
  labs(title="Expected Loss Rate by PD Risk Group",
       x="PD Group", y="EL Rate (%)") +
  theme_minimal()
```



15.4 Predicted EL vs Actual Loss (Back-test)

```
el_compare <- el_data |>
  group_by(pd_group) |>
  summarise(
    Predicted_EL = round(sum(EL)),
    Actual_Loss  = round(sum(AL)),
    .groups      = "drop"
  )

cat("========== PREDICTED EL vs ACTUAL LOSS (TEST SET) ==========\n")

## ========== PREDICTED EL vs ACTUAL LOSS (TEST SET) ==========

print(el_compare)

## # A tibble: 5 × 3
##   pd_group          Predicted_EL Actual_Loss
##   <fct>                    <dbl>       <dbl>
## 1 Very Low (<10%)       10341007    11212099
## 2 Low (10-20%)          18075563    18140332
## 3 Medium (20-30%)       12931928    12981656
## 4 High (30-50%)         12103171    11634559
## 5 Very High (>50%)       2947430     2571334

pivot_longer(el_compare, cols=c(Predicted_EL, Actual_Loss),
             names_to="Type", values_to="Amount") |>
```

```
  ggplot(aes(x=pd_group, y=Amount/1e6, fill=Type)) +
  geom_bar(stat="identity", position="dodge") +

scale_fill_manual(values=c("Predicted_EL"="steelblue","Actual_Loss"="#e74c3c"
),
                     labels =c("Predicted EL","Actual Loss")) +
  labs(title="Predicted Expected Loss vs Actual Loss (Test Set)",
       x="PD Group", y="Loss (millions $)", fill="") +
  theme_minimal()
```



15.5 Final Summary

```
cat("========== FINAL SUMMARY (TEST SET) ==========\n")

## ========== FINAL SUMMARY (TEST SET) ==========

cat(sprintf("Evaluated on       : Test set (%s loans)\n",
format(nrow(el_data), big.mark=",")))

## Evaluated on       : Test set (59,999 loans)

cat(sprintf("Total EAD          : $%s\n",   format(round(sum(el_data$EAD)),
big.mark=",")))

## Total EAD          : $919,275,525

cat(sprintf("Predicted EL       : $%s\n",   format(round(sum(el_data$EL)),
big.mark=",")))
```

```
## Predicted EL       : $56,399,097

cat(sprintf("Actual Loss        : $%s\n",   format(round(sum(el_data$AL)),
big.mark=",")))

## Actual Loss        : $56,539,980

cat(sprintf("EL / EAD           : %.2f%%\n", sum(el_data$EL) /
sum(el_data$EAD) * 100))

## EL / EAD           : 6.14%

cat(sprintf("AL / EAD (actual)  : %.2f%%\n", sum(el_data$AL) /
sum(el_data$EAD) * 100))

## AL / EAD (actual)  : 6.15%

cat(sprintf("Difference (EL-AL) : $%s (%.2f%%)\n",
            format(round(sum(el_data$EL) - sum(el_data$AL)), big.mark=","),
            (sum(el_data$EL) - sum(el_data$AL)) / sum(el_data$AL) * 100))

## Difference (EL-AL) : $-140,883 (-0.25%)
```